RATION MANAGEMENT SYSTEM

Complete Project Documentation — Compiled From Project Discussion

1. Overview

A system to manage the full lifecycle of ration supply for an institution running multiple messes: vendor procurement, central store inventory, distribution to messes, consumption logging, and demand forecasting that feeds back into the next procurement cycle. The defining property of the system is that it is a closed feedback loop, not a one-way pipeline: consumption data trains the forecast, and the forecast informs the next order.

```
Vendors -> Procurement -> Central Store -> Distribution -> Consumption
|  (data feeds up)                Dashboard / Reports / Audit Trail
|                    Demand Forecasting                              ^                 (past
consumption feeds forecast)                              |               (forecast guides
next order)
```

2. Roles & Permissions

Role	Access	Explicit exclusion
Administrator	All modules, all messes, user management, reports	—
Store Keeper	Inventory, Procurement receipt, Distribution	No budget/finance screens
Mess Manager	Consumption entry, own mess stock view	No other mess's data, no vendor-payment screen
Finance Staff	Budgeting, Billing, Reports	No inventory-edit actions
Vendor (Phase 2)	Order list, delivery status update	Everything else

Key rule: Mess Manager should never see the vendor-payment screen — enforced server-side, not just hidden in the UI.

3. Features

Feature	What it does
Order from vendors	Raise orders and record deliveries, even partial ones
Central stock tracking	Always-accurate stock, derived automatically from the transaction ledger
Send stock to messes	Move ration from central store to a mess with a full record
Daily usage entry	Each mess logs usage and headcount daily
Dashboard	One screen showing stock, spending, and alerts across all messes
Low-stock & expiry alerts	Warns before an item runs out or expires
Budget tracking	Spend vs. allotted budget
Audit trail	Every action recorded permanently, never editable

Feature	What it does
Demand forecasting	Predicts next cycle's requirement from past usage
Menu & feedback (optional)	Menu planning and mess-goer feedback collection
Vendor performance	Tracks on-time vs. late/short deliveries per vendor

4. Recommended Tech Stack

- Frontend: React, with role-aware routing
- Backend: Node.js + Express, or Django REST Framework
- Database: PostgreSQL (+ TimescaleDB extension later if needed)
- Cache: Redis, for dashboard aggregates and session data
- Auth: JWT with role + mess_id claims, short-lived access tokens with refresh rotation

Why PostgreSQL: native ACID transactions (so stock issues can't half-complete), Row-Level Security for role isolation enforced at the database level, and a mature extension ecosystem (TimescaleDB) for the time-series-heavy consumption and transaction data this system accumulates.

5. Database Schema — All Tables & Fields

14 tables total. stock_transaction is the core ledger — current stock is always calculated from it, never stored or hand-edited directly. audit_log is append-only, enforced at the database level via REVOKE UPDATE, DELETE.

Table	Fields
mess	id, name, location, headcount_capacity, created_at
app_user	id, name, email, password_hash, role, mess_id, is_active, created_at
vendor	id, name, contact_phone, contact_email, category, tax_id, is_active, created_at
item	id, name, unit, category, reorder_threshold, shelf_life_days, created_at
purchase_order	id, vendor_id, status, created_by, created_at, closed_at
purchase_order_line	id, po_id, item_id, ordered_qty, received_qty, unit_price
stock_transaction	id, type, item_id, quantity, source, destination, mess_id, po_id, reason_code, created_by, created_at
consumption_record	id, mess_id, item_id, date, quantity_used, headcount, recorded_by, created_at
budget	id, mess_id, period_start, period_end, allocated_amount, created_by, created_at
expenditure	id, po_id, vendor_id, amount, date, recorded_by, created_at
forecast_record	id, mess_id, item_id, period_start, period_end, predicted_qty, method, confidence, generated_at
menu	id, mess_id, date, meal, items, created_by
feedback	id, mess_id, menu_id, submitted_by, rating, comment, created_at

Table	Fields
audit_log	id, actor_id, action, entity_type, entity_id, before_value, after_value, created_at

6. API Design

6.1 Vendor & Procurement

Method	Endpoint	Purpose	Authorized roles
POST	/api/vendors	Create vendor record	Administrator
GET	/api/vendors	List vendors	Administrator, Store Keeper
GET	/api/vendors/:id/performance	Vendor delivery history	Administrator, Store Keeper
POST	/api/purchase-orders	Raise a PO against a vendor	Store Keeper, Administrator
PATCH	/api/purchase-orders/:id/receive	Record a delivery against a PO	Store Keeper
GET	/api/purchase-orders/:id	View PO detail and receipt history	Store Keeper, Administrator, Finance

6.2 Inventory & Distribution

Method	Endpoint	Purpose	Authorized roles
GET	/api/inventory/stock	Current derived stock levels	Administrator, Store Keeper
GET	/api/inventory/expiring	Items nearing expiry	Administrator, Store Keeper
POST	/api/inventory/adjustments	Record a stock adjustment with reason code	Store Keeper
POST	/api/distribution/issue	Issue stock from central store to a mess	Store Keeper
GET	/api/messes/:id/stock	Stock currently held at a specific mess	Mess Manager (own), Administrator

6.3 Consumption, Menu, Finance & Forecasting

Method	Endpoint	Purpose	Authorized roles
POST	/api/consumption	Log daily consumption for a mess	Mess Manager
GET	/api/consumption/:messId/history	Consumption history for a mess	Mess Manager (own), Administrator
POST / GET	/api/menu	Set or fetch a mess's menu	Mess Manager, Administrator

Method	Endpoint	Purpose	Authorized roles
POST / GET	/api/feedback	Submit or view mess feedback	Mess Manager, Administrator
GET	/api/reports/budget-vs-actual	Budget vs. spend report	Finance Staff, Administrator
GET	/api/reports/audit-log	Query the audit trail	Administrator, Finance Staff (read-only)
GET	/api/forecast/:messId/:itemId	Suggested order quantity	Administrator, Store Keeper

7. Role Dashboard Field Mapping

Admin Dashboard

Section	Field(s) used
Top summary bar	central_store_stock view; budget.allocated_amount vs expenditure.amount
Stock overview	central_store_stock.item_id, .quantity_on_hand, joined to item.name/.unit
Low-stock & expiry alerts	item.reorder_threshold, item.shelf_life_days vs stock_transaction.created_at
Mess drill-down	mess_stock.mess_id/.item_id/.quantity_on_hand; consumption_record.headcount/.date
Budget vs. spend	budget.mess_id/.allocated_amount/.period_start-end; expenditure.amount/.date
Vendor performance	vendor_performance view: .total_orders, .avg_fulfillment_pct, .late_orders
Audit trail (recent)	audit_log.actor_id/.action/.entity_type/.entity_id/.created_at
Forecast summary	forecast_record.mess_id/.item_id/.predicted_qty/.confidence
User management	app_user.name/.role/.mess_id/.is_active

Store Keeper Dashboard

Section	Field(s) used
Central store stock	central_store_stock.item_id/.quantity_on_hand, joined to item.*
Expiry watchlist	item.shelf_life_days, stock_transaction.created_at
Pending POs	purchase_order.status; purchase_order_line.ordered_qty/.received_qty
Quick actions (receive/issue)	writes stock_transaction.type/.item_id/.quantity/.po_id/.mess_id
Recent transactions	stock_transaction.type/.quantity/.created_at/.created_by
Vendor list	vendor.name/.contact_phone/.contact_email/.category

Mess Manager Dashboard

Section	Field(s) used
Today's entry form	writes consumption_record.item_id/.quantity_used/.headcount/.date
My mess's current stock	mess_stock.item_id/.quantity_on_hand, filtered to own mess_id
My consumption history	consumption_record.date/.quantity_used/.headcount, own mess_id
Menu (if enabled)	menu.date/.meal/.items, filtered to own mess_id
Feedback (if enabled)	feedback.rating/.comment/.created_at, filtered to own mess_id

Finance Staff Dashboard

Section	Field(s) used
Budget overview	budget.mess_id/.allocated_amount/.period_start-end vs summed expenditure.amount
Vendor payments	expenditure.vendor_id/.amount/.date, joined to vendor.name
PO financial view	purchase_order_line.unit_price x .ordered_qty/.received_qty
Audit trail (read-only)	same audit_log.* columns as Admin, no write path
Export	derived from the above, no new columns

Vendor Dashboard (Phase 2)

Section	Field(s) used
My orders	purchase_order.status/.created_at; purchase_order_line.ordered_qty/.received_qty — own vendor_id
Update delivery status	writes purchase_order.status, purchase_order_line.received_qty
My performance	vendor_performance view, scoped to own vendor_id

Pattern across all five: each dashboard's field list is just its role's permitted columns, filtered by mess_id or vendor_id where row-level security applies. Mess Manager and Vendor are the tightest-scoped; Admin has no row restriction; Finance never touches stock_transaction or consumption_record columns at all.

8. Diagrams Produced

Role	Write access	Read-only access
Store Keeper	Item, PurchaseOrder, PurchaseOrderLine, StockTransaction	Vendor
Mess Manager	ConsumptionRecord, Feedback	Mess (own), Menu
Finance Staff	Budget, Expenditure	PurchaseOrder, Vendor, AuditLog
Vendor (Phase 2)	PurchaseOrder, PurchaseOrderLine (own records only)	—

Role	Write access	Read-only access
Admin	AppUser	Every other table (Mess, Vendor, Item, PurchaseOrder, PurchaseOrderLine, StockTransaction, ConsumptionRecord, Budget, Expenditure, ForecastRecord, Menu, Feedback, AuditLog)

10. Implementation Plan

Week(s)	Phase	Deliverables
0	Foundations	Schema, migrations, auth + RBAC middleware, shared audit-log write path
1–2	Core inventory loop	Vendor + PO CRUD, receipt recording, stock ledger, distribution to messes
3	Consumption + dashboards	Consumption entry, admin/finance dashboards, low-stock alerts
4	Forecasting	Moving-average + seasonal model, suggested order quantities
5	Audit hardening	RLS policies, CSV/PDF export, load testing
6+	Vendor portal (Phase 2)	Separate threat model, external auth, order/delivery status UI